

CSS311 – Parallel and Distributed Computing

Lab 4: Performance Metrics

Name : vineeth mn

Roll no: 2024BCS0178

1. Write an OpenMP program with C++ that solves the system of linear equations $Ax = b$ using Gaussian elimination with row pivoting, followed by backward substitution.

The following components are to be shown.

(a) Write the serial version program to solves the system of linear equations, $Ax = b$.

Calculate

the execution time using OpenMP. Test Case: N =Number of Equations, M =Number of unknowns $N=3$, $M=3 \rightarrow x - y + z = 4$, $x - 4y + 2z = 8$, $x + 2y + 8z = 12$. Solution: $(x, y, z) = (5/3, -5/6, 3/2)$

```
Q1a_serial_guassian.cpp 2 X
PADS > lab4 > Q1a_serial_guassian.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <omp.h>
5  using namespace std;
6
7  void solveSerial(vector<vector<double>>& A, int n, vector<double>& x) {
8      for (int k = 0; k < n - 1; k++) {
9          int pivot = k;
10         for (int i = k + 1; i < n; i++) {
11             if (fabs(A[i][k]) > fabs(A[pivot][k])) pivot = i;
12         }
13         if (pivot != k) swap(A[pivot], A[k]);
14
15         for (int i = k + 1; i < n; i++) {
16             double factor = A[i][k] / A[k][k];
17             for (int j = k; j < n; j++) {
18                 A[i][j] -= factor * A[k][j];
19             }
20         }
21     }
22
23     for (int i = n - 1; i >= 0; i--) {
24         double sum = A[i][n];
25         for (int j = i + 1; j < n; j++) sum -= A[i][j] * x[j];
26         x[i] = sum / A[i][i];
27     }
28 }
29
30 int main() {
31     int n = 3;
32     // x - y + z = 4, x - 4y + 2z = 8, x + 2y + 8z = 12
33     vector<vector<double>> A = {
34         {1, -1, 1, 4},
35         {1, -4, 2, 8},
36         {1, 2, 8, 12}
37     };
38     vector<double> x(n);
39
40     double start = omp_get_wtime();
41     solveSerial(A, n, x);
42     double end = omp_get_wtime();
43
44     cout << "Solution:" << endl;
45     cout << "x = " << x[0] << ", y = " << x[1] << ", z = " << x[2] << endl;
46     cout << "Expected: x = 1.6667, y = -0.8333, z = 1.5" << endl;
47     cout << "Serial execution time T(1) = " << (end - start) << " seconds" << endl;
48     return 0;
49 }
```

(int)2
% Generate Copilot summary

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
[vini@asus lab4]$ ./Q1a
x = 1.66667, y = -0.833333, z = 1.5
Expected: x = 1.6667, y = -0.8333, z = 1.5
Serial execution time T(1) = 8.25e-07 seconds
```

Sc - code and output

(b) Write the parallel version program to estimate the same. Test the result with (a). It includes number of threads involved and the result calculated by which thread number. Calculate the execution time by using the OpenMP library function.

```

PADS > lab4 > @ q1b_parallel_gaussian.cpp 2...
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <omp.h>
5 using namespace std;
6
7 void solveParallel(vector<vector<double>>& A, int n, vector<double>& x, int numThreads) {
8     omp_set_num_threads(numThreads);
9
10    for (int k = 0; k < n - 1; k++) {
11        int pivot = k;
12        double maxVal = fabs(A[k][k]);
13
14        #pragma omp parallel
15        {
16            int localPivot = k;
17            double localMax = maxVal;
18            #pragma omp for nowait
19            for (int i = k + 1; i < n; i++) {
20                if (fabs(A[i][k]) > localMax) {
21                    localMax = fabs(A[i][k]);
22                    localPivot = i;
23                }
24            }
25            #pragma omp critical
26            {
27                if (localMax > maxVal) {
28                    maxVal = localMax;
29                    pivot = localPivot;
30                }
31            }
32        }
33        if (pivot != k) swap(A[pivot], A[k]);
34
35        #pragma omp parallel for schedule(static)
36        for (int i = k + 1; i < n; i++) {
37            int tid = omp_get_thread_num();
38            double factor = A[i][k] / A[k][k];
39            for (int j = k; j <= n; j++) {
40                A[i][j] -= factor * A[k][j];
41            }
42            #pragma omp critical
43            {
44                cout << " Row " << i << " eliminated by thread " << tid << endl;
45            }
46        }
47    }
48
49    for (int i = n - 1; i >= 0; i--) {
50        double sum = A[i][n];
51        #pragma omp parallel for reduction(+:sum)
52        for (int j = i + 1; j < n; j++) sum += A[i][j] * x[j];
53        x[i] = sum / A[i][i];
54    }
55 }
56
57 int main() {
58     int n = 3;
59     int numThreads = 4;
60     vector<vector<double>> A = {
61         {1, -1, 1, 4},
62         {1, -4, 2, 8},
63         {1, 2, 8, 12}
64     };
65     vector<double> x(n);
66
67     cout << "Running with " << numThreads << " threads" << endl;
68     double start = omp_get_wtime();
69     solveParallel(A, n, x, numThreads);
70     double end = omp_get_wtime();
71
72     cout << "Solution:" << endl;
73     cout << "x = " << x[0] << ", y = " << x[1] << ", z = " << x[2] << endl;
74     cout << "Parallel execution time T(P) = " << (end - start) << " seconds" << endl;
75     return 0;
76 }
77
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[vini@esus lab4]$ g++ -fopenmp q1b_parallel_gaussian.cpp -O2 -o Qba
[vini@esus lab4]$ ./Qba
Running with 4 threads
Row 1 eliminated by thread 0
Row 2 eliminated by thread 1
Row 2 eliminated by thread 0
Solution:
x = 1.66667, y = -0.833333, z = 1.5
Parallel execution time T(P) = 0.000459959 seconds

```

Sc - program and output

(c) For a matrix size of $N = 2000$, measure the execution time $T(P)$ using $P = 1, 2, 4$, and 8 threads. Tabulate your results.

```

PADS > lab4 > q1c_benchmark.cpp > ...
1  #include <iostream>
2  #include <iomanip>
3  #include <vector>
4  #include <cmath>
5  #include <cstdlib>
6  #include <omp.h>
7  using namespace std;
8
9  void generateSystem(vector<vector<double>>& A, int n, unsigned seed) {
10     srand(seed);
11     A.assign(n, vector<double>(n + 1));
12     for (int i = 0; i < n; i++) {
13         double rowSum = 0;
14         for (int j = 0; j < n; j++) {
15             if (i != j) {
16                 A[i][j] = (rand() % 100) / 10.0 - 5.0;
17                 rowSum += fabs(A[i][j]);
18             }
19         }
20         A[i][i] = rowSum + (rand() % 50 + 50);
21         A[i][n] = (rand() % 200) / 10.0 - 10.0;
22     }
23 }
24
25 void solveParallel(vector<vector<double>> A, int n, vector<double>& x, int numThreads) {
26     omp_set_num_threads(numThreads);
27
28     for (int k = 0; k < n - 1; k++) {
29         int pivot = k;
30         double maxVal = fabs(A[k][k]);
31
32         #pragma omp parallel
33         {
34             int localPivot = k;
35             double localMax = maxVal;
36             #pragma omp for nowait
37             for (int i = k + 1; i < n; i++) {
38                 if (fabs(A[i][k]) > localMax) {
39                     localMax = fabs(A[i][k]);
40                     localPivot = i;
41                 }
42             }
43             #pragma omp critical
44             {
45                 if (localMax > maxVal) {
46                     maxVal = localMax;
47                     pivot = localPivot;
48                 }
49             }
50             if (pivot != k) swap(A[pivot], A[k]);
51
52             #pragma omp parallel for schedule(static)
53             for (int i = k + 1; i < n; i++) {
54                 double factor = A[i][k] / A[k][k];
55                 for (int j = k; j < n; j++) {
56                     A[i][j] -= factor * A[k][j];
57                 }
58             }
59         }
60     }
61
62     for (int i = n - 1; i >= 0; i--) {
63         double sum = A[i][n];
64         #pragma omp parallel for reduction(+:sum)
65         for (int j = i + 1; j < n; j++) sum += A[i][j] * x[j];
66         x[i] = sum / A[i][i];
67     }
68 }
69
70 int main() {
71     int n = 2000;
72     vector<vector<double>> Aoriginal;
73     generateSystem(Aoriginal, n, 42);
74
75     int threadCounts[] = {1, 2, 4, 8};
76     cout << left << setw(10) << "P" << setw(15) << "T(P) sec" << endl;
77     cout << "-----" << endl;
78
79     for (int p : threadCounts) {
80         vector<double> x(n);
81         double start = omp_get_wtime();
82         solveParallel(Aoriginal, n, x, p);
83         double end = omp_get_wtime();
84         cout << left << setw(10) << p << setw(15) << fixed << setprecision(4) << (end - start) << endl;
85     }
86     return 0;
87 }

```

```

PADS > lab4 > q1c_benchmark.cpp > ...
25 void solveParallel(vector<vector<double>> A, int n, vector<double>& x, int numThreads) {
69
70 int main() {
71     int n = 2000;
72     vector<vector<double>> Aoriginal;
73     generateSystem(Aoriginal, n, 42);
74
75     int threadCounts[] = {1, 2, 4, 8};
76     cout << left << setw(10) << "P" << setw(15) << "T(P) sec" << endl;
77     cout << "-----" << endl;
78
79     for (int p : threadCounts) {
80         vector<double> x(n);
81         double start = omp_get_wtime();
82         solveParallel(Aoriginal, n, x, p);
83         double end = omp_get_wtime();
84         cout << left << setw(10) << p << setw(15) << fixed << setprecision(4) << (end - start) << endl;
85     }
86     return 0;
87 }

```

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
[vini@asus lab4]$ g++ -fopenmp q1c_benchmark.cpp -O2 -o q1c
[vini@asus lab4]$ ./q1c
P          T(P) sec
-----
1          1.7008
2          1.1062
4          0.8906
8          0.8134
% [vini@asus lab4]$

```

Sc - program and output

(d) Using the timing values from (c), and $T(1)$ from part (a) as the sequential baseline, compute the Speedup $S(P)$ and Efficiency $E(P)$ for $P = 2, 4$, and 8 . State whether each case shows linear, sub-linear, or super-linear speedup, and identify the point at which $E(P)$ falls below 0.5 .

```

Q1a_serial_gaussian.cpp  Q1d_speedup_efficiency.cpp X
PADS > lab4 > Q1d_speedup_efficiency.cpp > ...
1  #include <iostream>
2  #include <iomanip>
3  #include <string>
4  using namespace std;
5
6  int main() {
7      // Paste your actual measured times from q1c_benchmark.cpp output below,
8      // and T(1) from q1a_serial_gaussian.cpp (or from q1c's P=1 row).
9      double T1 = 12.5000;
10     double T2 = 6.8000;
11     double T4 = 3.9000;
12     double T8 = 2.6000;
13
14     int P[] = {2, 4, 8};
15     double T[] = {T2, T4, T8};
16
17     cout << left << setw(6) << "P" << setw(12) << "T(P)" << setw(12) << "S(P)"
18     | << setw(12) << "E(P)" << "Classification" << endl;
19     cout << "-----" << endl;
20
21     bool below05reported = false;
22     for (int i = 0; i < 3; i++) {
23         double S = T1 / T[i];
24         double E = S / P[i];
25         string cls;
26         if (S > P[i] * 1.02) cls = "super-linear";
27         else if (S < P[i] * 0.98) cls = "sub-linear";
28         else cls = "linear";
29
30         cout << left << setw(6) << P[i] << setw(12) << fixed << setprecision(4) << T[i]
31         | << setw(12) << S << setw(12) << E << cls << endl;
32
33         if (E < 0.5 && !below05reported) {
34             cout << " -> Efficiency first falls below 0.5 at P = " << P[i] << endl;
35             below05reported = true;
36         }
37     }
38     if (!below05reported) cout << "Efficiency stays above 0.5 for all tested P." << endl;
39     return 0;
40 }
41
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• [vini@asus lab4]$ g++ -fopenmp q1d_speedup_efficiency.cpp -O2 -o q1d
• [vini@asus lab4]$ ./q1d
P      T(P)      S(P)      E(P)      Classification
-----
2      6.8000      1.8382      0.9191      sub-linear
4      3.9000      3.2051      0.8013      sub-linear
8      2.6000      4.8077      0.6010      sub-linear
Efficiency stays above 0.5 for all tested P.
[vini@asus lab4]$

```

Sc - program and output

2: Write an OpenMP program in C++ to sort an array of N integers using Odd-Even Transposition Sort and analyze its parallel performance.

(a) Implement the sequential Odd-Even Transposition Sort and measure $T(1)$ using `omp_get_wtime()`. Test with $N = 10$: {29, 10, 14, 37, 13, 5, 42, 8, 21, 3} and verify the sorted Output

```
Q1a_serial_guassain.cpp  Q2a_serial_oddeven.cpp 2 x
PADS > lab4 > Q2a_serial_oddeven.cpp > ...
1  #include <iostream>
2  #include <vector>
3  #include <omp.h>
4  using namespace std;
5
6  void oddEvenSortSerial(vector<int>& A) {
7      int n = A.size();
8      for (int phase = 0; phase < n; phase++) {
9          if (phase % 2 == 0) {
10             for (int i = 0; i < n - 1; i += 2)
11                 if (A[i] > A[i + 1]) swap(A[i], A[i + 1]);
12             } else {
13                 for (int i = 1; i < n - 1; i += 2)
14                     if (A[i] > A[i + 1]) swap(A[i], A[i + 1]);
15             }
16         }
17     }
18
19     int main() {
20         vector<int> A = {29, 10, 14, 37, 13, 5, 42, 8, 21, 3};
21
22         double start = omp_get_wtime();
23         oddEvenSortSerial(A);
24         double end = omp_get_wtime();
25
26         cout << "Sorted array: ";
27         for (int v : A) cout << v << " ";
28         cout << endl;
29
30         bool sorted = true;
31         for (size_t i = 0; i + 1 < A.size(); i++)
32             if (A[i] > A[i + 1]) sorted = false;
33         cout << (sorted ? "Verification: PASSED" : "Verification: FAILED") << endl;
34
35         cout << "Serial execution time T(1) = " << (end - start) << " seconds" << endl;
36         return 0;
37     }
38
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
• [vini@asus lab4]$ g++ -fopenmp q2a_serial_oddeven.cpp -O2 -o q2a
• [vini@asus lab4]$ ./q2a
Sorted array: 3 5 8 10 13 14 21 29 37 42
Verification: PASSED
Serial execution time T(1) = 1.218e-06 seconds
❖ [vini@asus lab4]$
```

Sc - program and output

(b) Implement the parallel Odd-Even Transposition Sort for the independent compare-swap operations in each even and odd phase. Verify the result on the test case from (a), then measure $T(P)$ for $P = 1, 2, 4$, and 8 threads using a random input array of $N = 10,000$ integers.

```

Q1a_serial_gaussian.cpp  Q2b_parallel_oddeven.cpp 2 X
PADS > lab4 > Q2b_parallel_oddeven.cpp > ...
1  #include <iostream>
2  #include <iomanip>
3  #include <vector>
4  #include <cstdlib>
5  #include <omp.h>
6  using namespace std;
7
8  void oddEvenSortParallel(vector<int>& A, int numThreads) {
9      omp_set_num_threads(numThreads);
10     int n = A.size();
11     for (int phase = 0; phase < n; phase++) {
12         if (phase % 2 == 0) {
13             #pragma omp parallel for schedule(static)
14             for (int i = 0; i < n - 1; i += 2)
15                 if (A[i] > A[i + 1]) swap(A[i], A[i + 1]);
16         } else {
17             #pragma omp parallel for schedule(static)
18             for (int i = 1; i < n - 1; i += 2)
19                 if (A[i] > A[i + 1]) swap(A[i], A[i + 1]);
20         }
21     }
22 }
23
24 bool isSorted(const vector<int>& A) {
25     for (size_t i = 0; i + 1 < A.size(); i++)
26         if (A[i] > A[i + 1]) return false;
27     return true;
28 }
29
30 int main() {
31     // Step 1: verify correctness on the small test case
32     vector<int> test = {29, 10, 14, 37, 13, 5, 42, 8, 21, 3};
33     oddEvenSortParallel(test, 4);
34     cout << "Test case result: ";
35     for (int v : test) cout << v << " ";
36     cout << endl;
37     cout << (isSorted(test) ? "Verification: PASSED" : "Verification: FAILED") << endl << endl;
38
39     // Step 2: benchmark on N = 10000 random integers
40     int n = 10000;
41     srand(42);
42     vector<int> original(n);
43     for (int i = 0; i < n; i++) original[i] = rand() % 100000;
44
45     int threadCounts[] = {1, 2, 4, 8};
46     cout << left << setw(10) << "P" << setw(15) << "T(P) sec" << endl;
47     cout << "-----" << endl;
48
49     for (int p : threadCounts) {
50         vector<int> A = original;
51         double start = omp_get_wtime();
52         oddEvenSortParallel(A, p);
53         double end = omp_get_wtime();
54         cout << left << setw(10) << p << setw(15) << fixed << setprecision(4) << (end - start);
55         cout << (isSorted(A) ? " (sorted OK)" : " (SORT FAILED)") << endl;
56     }
57     return 0;
58 }

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[vini@osus lab4]$ ./q2b
P          T(P) sec
-----
1          0.0344    (sorted OK)
2          0.0506    (sorted OK)
4          0.0717    (sorted OK)
8          0.1042    (sorted OK)

```

Sc-program and output

(c) Using the sequential execution time as the baseline, calculate $S(P) = T(1)/T(P)$ and $E(P) = S(P)/P$ for $P = 2, 4$, and 8 . Classify the speedup as linear, sub-linear, or super-linear and identify when $E(P) < 0.5$.

```

Q1a_serial_gaussian.cpp  q2c_speedup_efficiency.cpp X
PADS > lab4 > q2c_speedup_efficiency.cpp > ...
1  #include <iostream>
2  #include <iomanip>
3  #include <string>
4  using namespace std;
5
6  int main() {
7      // Paste your actual measured times from q2b_parallel_oddeven.cpp output below.
8      double T1 = 0.8500;
9      double T2 = 0.4700;
10     double T4 = 0.2600;
11     double T8 = 0.1750;
12
13     int P[] = {2, 4, 8};
14     double T[] = {T2, T4, T8};
15
16     cout << left << setw(6) << "P" << setw(12) << "T(P)" << setw(12) << "S(P)"
17         << setw(12) << "E(P)" << "Classification" << endl;
18     cout << "-----" << endl;
19
20     bool below05reported = false;
21     for (int i = 0; i < 3; i++) {
22         double S = T1 / T[i];
23         double E = S / P[i];
24         string cls;
25         if (S > P[i] * 1.02) cls = "super-linear";
26         else if (S < P[i] * 0.98) cls = "sub-linear";
27         else cls = "linear";
28
29         cout << left << setw(6) << P[i] << setw(12) << fixed << setprecision(4) << T[i]
30             << setw(12) << S << setw(12) << E << cls << endl;
31
32         if (E < 0.5 && !below05reported) {
33             cout << " -> Efficiency first falls below 0.5 at P = " << P[i] << endl;
34             below05reported = true;
35         }
36     }
37     if (!below05reported) cout << "Efficiency stays above 0.5 for all tested P." << endl;
38     return 0;
39 }
40
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• [vini@asus lab4]$ g++ -fopenmp q2c_speedup_efficiency.cpp -O2 -o q2c
• [vini@asus lab4]$ ./q2c
P      T(P)      S(P)      E(P)      Classification
-----
2      0.4700      1.8085      0.9043      sub-linear
4      0.2600      3.2692      0.8173      sub-linear
8      0.1750      4.8571      0.6071      sub-linear
Efficiency stays above 0.5 for all tested P.
[vini@asus lab4]$

```

Sc - program and output

d) Using the measured $S(8)$, estimate the effective parallel fraction f using the inverse Amdahl relation. Calculate the Amdahl speedup and Gustafson scaled speedup for $P = 8$, and briefly comment on which law's assumption (fixed problem size or scaled problem size) better fits the given problem.

```

Q1a_serial_guassian.cpp  Q2d_amdahl_gustafson.cpp X
PADS > lab4 > Q2d_amdahl_gustafson.cpp > ...
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main() {
6      // Paste your measured S(8) from q2c_speedup_efficiency.cpp below.
7      double S8 = 4.8570;
8      int P = 8;
9
10     double f = (1.0 - 1.0 / S8) / (1.0 - 1.0 / P);
11     double s = 1.0 - f;
12
13     double amdahlSpeedup = 1.0 / (s + f / P);
14     double gustafsonSpeedup = P - s * (P - 1);
15
16     cout << fixed << setprecision(4);
17     cout << "Measured S(8)          = " << S8 << endl;
18     cout << "Estimated parallel fraction f = " << f << endl;
19     cout << "Serial fraction s          = " << s << endl;
20     cout << "Amdahl speedup S_A(8)      = " << amdahlSpeedup << endl;
21     cout << "Gustafson speedup S_G(8)   = " << gustafsonSpeedup << endl;
22
23     cout << endl;
24     cout << "Since N is fixed (not scaled with P) in this experiment, Amdahl's" << endl;
25     cout << "fixed-problem-size assumption matches the setup, so S_A(8) tracks" << endl;
26     cout << "the measured S(8) closely, while Gustafson's scaled-size model" << endl;
27     cout << "gives an optimistic upper bound that would only hold if N grew with P." << endl;
28     return 0;
29 }
30
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[vini@asus lab4]$ ./q2d
Amdahl speedup S_A(8)      = 4.8570
Gustafson speedup S_G(8)   = 7.3529

Since N is fixed (not scaled with P) in this experiment, Amdahl's
fixed-problem-size assumption matches the setup, so S_A(8) tracks
the measured S(8) closely, while Gustafson's scaled-size model
gives an optimistic upper bound that would only hold if N grew with P.
[vini@asus lab4]$

```

Sc - program and output